PAPPO: Patch-Aware PPO for Tool-Using Coding Agents

Technical report, v0.1.0-trueppo

Abstract

Tool-using coding agents operate over long, sparse-reward trajectories: an agent may read files, search, edit code, and run tests before receiving a final success signal. Applying the same terminal reward to every generated token or turn gives a weak training signal, while supervised LoRA weighting cannot directly express policy improvement against the behavior policy. PAPPO introduces patch-aware, turn-local credit assignment for repository repair and embeds it in a true PPO loop over tool-call action tokens. The implementation stores old-policy and reference-policy log probabilities, optimizes an action-masked clipped PPO objective with KL/reference control, uses grouped turn-level scalar baselines, applies cumulative LoRA updates, and re-rolls out after every update.

On a local RealRepoFix-100 feasibility study with `Qwen/Qwen2.5-Coder-7B-Instruct`, PAPPO-PPO improves mean held-out success from 0.8667 for the best non-PPO baseline to 1.0000 across three seeds, reduces the failed-edit rate from 0.1333 to 0.0000, and keeps average tool cost unchanged at 8.50. This result supports a narrow claim: turn-local PAPPO credit assignment is useful when placed inside a real PPO update loop for this model and benchmark. It is not yet evidence of broad generalization across coding agents, models, or benchmarks.

1. Motivation

Repository-level coding agents are not ordinary single-response language models. They make sequences of tool decisions: inspect a file, locate a failing test, generate a patch, run tests, and sometimes retry. The final outcome is easy to measure, but it is too coarse to explain which local action helped. This creates two related problems for reinforcement learning from coding-agent trajectories.

First, trace-level rewards assign the same return to many heterogeneous actions. A useful search, a harmful edit, and a redundant test can all receive the same label if they appear in the same trajectory. Second, reward-weighted supervised fine-tuning can emphasize high-reward traces, but it does not compare the new policy against the policy that produced the sampled action. It therefore lacks the central PPO mechanism: a controlled policy-ratio update on sampled actions.

PAPPO is designed around the observation that coding-agent trajectories contain local evidence. An edit tool result can tell whether a patch was applied. Test output can tell whether a patch moved the task toward success. Search output can indicate whether the agent found relevant test context. PAPPO converts this local evidence into turn-level targets and uses those targets as advantages in a true PPO update.

2. Know Why

PAPPO starts from a practical failure mode in coding-agent training. A repair trajectory may end with a binary success signal, but the trajectory itself is made of qualitatively different decisions. A search turn may be useful even if the later edit is wrong. An edit may apply cleanly but still fail tests. A test run may reveal progress without being the final successful action. Treating all of those turns as equally good or equally bad loses the information that makes repository repair learnable.

The first design choice is therefore to make credit assignment patch-aware. The algorithm should care about whether an edit produced an actual patch, whether that patch was associated with final success, and whether surrounding tool evidence indicates progress or wasted effort. This gives a sharper signal than trace reward broadcasting.

The second design choice is to keep PPO as the optimization mechanism rather than replacing it with reward-weighted supervised fine-tuning. Weighted LoRA can say "imitate this high-scoring action more", but it does not measure how far the new policy has moved from the policy that sampled the action. PPO contributes old-policy

ratios, clipping, and KL/reference control, which are exactly the mechanisms needed when online rollouts change after each update.

The third design choice is to train on turns instead of whole conversations. For coding agents, the meaningful unit of intervention is often a tool call: what file to edit, what patch text to emit, whether to run tests. PAPPO turns those local tool calls into PPO samples. The result is a method whose purpose is not just to make the model "more successful" in aggregate, but to make the edit actions that actually affect repository state more reliable.

This is the "know why" behind the project:

- Terminal rewards are too coarse for repository repair.
- Static reward-weighted LoRA is too weak to express controlled policy

improvement.

- Tool-result metadata contains local evidence that should not be discarded.
- PPO becomes more useful for coding agents when its advantages are attached to

patch-aware turns rather than broadcast over entire traces.

3. Problem Setup

We consider a tool-using coding agent policy `pi_theta` that generates tool-call actions during a repository repair trajectory. A trajectory contains a task description, a sequence of tool turns, tool results, and a final task reward. The goal is to improve held-out repair success while avoiding degenerate behavior such as failed edits, repeated tests, or higher tool cost.

The experiment focuses on edit actions because repository repair success is ultimately mediated by patches. For each generated edit action, PAPPO records:

- the prompt prefix that led to the action,
- the generated action text,
- a patch-aware turn target,
- a scalar baseline value,
- old-policy token log probabilities,
- reference-policy token log probabilities, and
- an action mask that restricts optimization to response/action tokens.

This yields a batch of turn samples rather than a single scalar label for the whole trace.

4. Method

4.1 Patch-Aware Turn Targets

PAPPO assigns local targets using tool-result metadata and final task outcome. For edit turns, an applied patch in a successful trajectory receives positive credit, an applied patch in a failed trajectory receives negative credit, and a failed edit receives a smaller negative target. Test and search turns can also receive local signals when their metadata exposes test progress or useful context. In the current true-PPO experiment, only edit turns are optimized, but the scoring function is trajectory-aware and tool-aware.

This differs from trace reward broadcasting. A trace reward asks, "Did the whole trajectory succeed?" PAPPO asks, "Given the local tool result and the final outcome, did this specific turn look useful?"

4.2 Turn-Level Baseline And Advantage

Each PPO sample has a scalar target `y_t` and a baseline value `V_t`. The scalar advantage is:

```
A_t = y_t - V_t
```

The released RealRepoFix-100 run uses grouped turn-level baselines and a small local prior. The prior folds part of the local target into the value term before advantage computation, increasing the influence of local patch evidence while still retaining a baseline. Advantage normalization is disabled in the final reported run because the unnormalized signal was more stable for this small turn-level batch setting.

4.3 True PPO Over Action Tokens

For every sampled action, PAPPO stores old-policy log probabilities before the update. It also stores reference-policy log probabilities from the base model or the configured reference adapter. During training, the current policy produces new log probabilities for the same action tokens. PPO then optimizes only the action span, not the prompt tokens.

For action-token mask `m`, old log probability `log pi_old`, new log probability `log pi_theta`, and advantage `A`, the clipped policy term is:

```
r_t = exp(log pi_theta - log pi_old)
L_clip = -mean_m(min(r_t A, clip(r_t, 1 - eps, 1 + eps) A))
```

PAPPO also adds a KL/reference control term:

```
delta_ref = log pi_theta - log pi_ref
KL_ref = mean_m(exp(delta_ref) - 1 - delta_ref)
```

and a scalar value loss:

```
L_value = mse(V_t, y_t)
```

The final per-sample loss is:

```
L = L_clip + beta * KL_ref + c_v * L_value
```

The implementation trains LoRA adapters cumulatively. After each PPO update, the agent rolls out again with the updated adapter, extracts fresh samples, stores new old-policy log probabilities, and evaluates the checkpoint on held-out tasks.

4.4 Difference From Traditional PPO

PAPPO is not a replacement for PPO's clipped objective. It is a coding-agent adaptation of PPO's credit-assignment interface.

Traditional PPO usually receives environment returns or learned rewards for actions in a Markov decision process. In many language-agent settings, the reward is attached to the full generated response or the full trace, then broadcast over tokens. PAPPO changes the reward construction and sample shape:

- The optimized unit is a tool-call turn, especially an edit action, rather than

a whole conversation.

- The reward target is patch-aware and tool-result-aware, not only terminal

success.

- PPO is masked to action tokens so the update focuses on generated tool-call

content.

- Baselines are grouped at the turn/tool level to reduce coarse trace-level

variance.

- Fresh rollouts after updates keep the training loop online instead of turning

the method into static reward-weighted SFT.

In short, PPO supplies the stable policy-improvement machinery; PAPPO supplies the local coding-agent credit signal.

5. Project Architecture

The repository is organized around a small number of explicit boundaries. The goal is that a reader can trace the full experiment from task definition to agent rollout, PPO sample construction, LoRA update, evaluation, and result audit.

5.1 Runtime Objects

The core data object is an `AgentTrajectory`. It records the task, tool calls, tool results, metadata, and final reward for one repair attempt. Tool-call turns are split out of the trajectory so the PPO code can operate on the action that the model generated, not on the entire conversation transcript.

A PPO training unit is a `PPOTurnSample`. It contains one optimized edit action: the prompt, generated action text, local target, baseline value, old logprobs, reference logprobs, and action mask. This object is the bridge between agent rollout code and PPO optimization.

5.2 Main Modules

The main source files have the following roles:

- `pappo/trajectory.py`: trajectory and turn representations.
- `pappo/realrepofix.py`: RealRepoFix task loading and repository repair task

representation.

- `pappo/llm_agent_pilot.py`: tool-using agent runtime over model backends.
- `pappo/lora_training.py`: model loading, LoRA helpers, and patch-aware local

scoring functions.

- `pappo/ppo_rollout.py`: conversion from trajectories to `PPOTurnSample`

records.

- `pappo/turn_critic.py`: simple tool-level and grouped turn-level scalar

critics.

- `pappo/ppo_training.py`: old/ref logprob handling, action-token masking,

clipped PPO loss, KL/reference control, and LoRA adapter updates.

- `scripts/run_realrepo_pappo_ppo.py`: end-to-end true-PPO experiment runner.
- `scripts/run_realrepo_lora_comparison.py`: deterministic non-PPO baseline

runner and shared evaluation utilities.

The important architectural separation is that rollout code does not know how to optimize PPO, and PPO code does not know how to repair repositories. They communicate through serialized trajectories and `PPOTurnSample` records.

5.3 End-To-End Data Flow

The final experiment follows this flow:

```
RealRepoFix manifest
  -&gt; train/eval task split
  -&gt; coding-agent rollouts
  -&gt; AgentTrajectory records
  -&gt; edit-turn PPOTurnSample records
  -&gt; grouped turn critic values
  -&gt; old-policy and reference-policy logprobs
  -&gt; clipped action-masked PPO LoRA update
  -&gt; updated adapter checkpoint
  -&gt; held-out eval rollouts
  -&gt; report.json and audit metrics
```

Each PPO update repeats the rollout and sample-construction stages using the latest adapter. This is why the method is online in the practical sense used by this report: training samples are refreshed after policy updates instead of being fixed once at the start.

5.4 Artifact Layout

The reproducibility package intentionally keeps only the artifacts needed to audit the result:

- `data/realrepofix_100_manifest.jsonl` and `data/realrepofix_100/` define the benchmark tasks.
- `data/realrepo_lora_comparison_100_seed*_deterministic_v2/report.json` contain the non-PPO baseline reports.
- `data/realrepo_pappo_ppo_100_seed*_grouped_prior_lr1e4_nonorm_updates5*/report.json` contain the PAPPO-PPO reports.
- `docs/pappo_true_ppo_results.md` summarizes the result gate.
- `docs/pappo_technical_report.md` is the narrative technical report.

Generated adapters, large model weights, rollout dumps, and exploratory experiment directories are ignored so the repository remains a focused reproducibility package rather than a local training cache.

6. Know How

PAPPO can be understood as an implementation recipe as much as an algorithmic idea. The following sequence is the practical know-how needed to reproduce or extend the method.

6.1 Build A Repair Task Set

Start with a manifest of repository repair tasks. Each task should expose enough structure for the agent to inspect files, edit code, and run tests. The RealRepoFix-100 package provides this through `data/realrepofix_100_manifest.jsonl`.

For credible evaluation, split tasks before training. The reported runs use 70 train tasks and 30 held-out eval tasks per seed. The held-out tasks are never used for PPO updates.

6.2 Roll Out The Current Agent

Run the current policy on the train tasks. For each task, keep the full trajectory: prompts, tool calls, tool results, metadata, and final reward. The reported setting uses two stochastic rollouts per train task per update. This gives the grouped critic local alternatives to compare within the same task family.

6.3 Extract Patch-Aware PPO Samples

Convert trajectories into edit-turn PPO samples. Each sample should preserve the exact prompt prefix and generated action text. Then compute the local target from patch and test metadata. This is the point where PAPPO differs most from ordinary trace-level RL: the training target is attached to the edit action that changed repository state.

6.4 Fit A Turn Baseline

Fit a scalar baseline for the collected samples. The released run uses grouped turn means with a tool-level fallback. The baseline is intentionally simple; its job is to reduce variance and encode local comparison structure, not to become a large learned reward model.

6.5 Freeze Old And Reference Logprobs

Before updating the adapter, compute token log probabilities for the sampled actions under the current policy. These are the old-policy logprobs used in the PPO ratio. Also compute reference logprobs from the base/reference policy for KL control. This step is what separates the true PPO loop from reward-weighted SFT.

6.6 Update The LoRA Adapter

Train the adapter with the action-masked clipped PPO objective. The action mask keeps prompt tokens out of the policy loss. The final experiment uses one PPO epoch per update, learning rate `1e-4`, KL beta `0.01`, clip epsilon `0.2`, and disabled advantage normalization.

6.7 Evaluate, Then Repeat

After each update, run held-out evaluation with the current adapter and record success rate, failed edit rate, average tool cost, and repeated test rate. Then use the updated adapter for the next train rollout. The loop is:

```
roll out -&gt; score turns -&gt; fit baseline -&gt; freeze logprobs -&gt; PPO update
         -&gt; evaluate checkpoint -&gt; roll out again
```

The key operational rule is to avoid mixing evaluation data into PPO samples. Held-out eval is only for measurement.

6.8 Debugging Checklist

If PAPPO does not improve, the most useful checks are:

- Confirm that `old_logprobs` are computed before the update and are not accidentally recomputed after training.
- Confirm that the action mask covers generated action tokens and not prompt tokens.
- Inspect the distribution of local targets; all-zero or single-valued targets remove the advantage signal.
- Compare checkpoint success across updates rather than trusting only the final adapter.
- Watch KL. A flat KL with no metric movement can indicate too little learning; a large KL jump can indicate unstable adapter drift.
- Audit auxiliary behavior. A higher success rate is less meaningful if it only comes from more tool calls, repeated tests, or unsafe failed edits.

7. Experimental Setup

7.1 Model And Benchmark

The released experiment uses:

- Model: `Qwen/Qwen2.5-Coder-7B-Instruct`
- Benchmark: RealRepoFix-100
- Per-seed split: 70 train tasks and 30 held-out eval tasks
- Seeds: 0, 1, 2
- PPO updates: 5
- Train rollouts: 2 stochastic rollouts per train task per update
- Eval decoding: stochastic, temperature 0.7
- LoRA learning rate: `1e-4`
- PPO epochs per update: 1
- Local PAPPO prior: 0.25
- Advantage normalization: disabled
- Max new tokens: 384

- Max training length: 768

The comparison uses the best non-PPO baseline from deterministic weighted-LoRA reports, including the base model and non-PPO reward-weighted variants. The reported PAPPO result uses the true PPO loop described above.

7.2 Metrics

The main metric is held-out task success rate. Auxiliary metrics track whether the improvement comes with undesirable behavior:

- failed edit rate,
- average tool cost, and
- repeated test rate.

The desired behavior is higher success without increasing these auxiliary costs.

8. Results

8.1 Three-Seed Held-Out Result

Seed	Best Non-PPO Success	PAPPO-PPO Base	PAPPO-PPO Final	Delta	Failed Edit: Best -> PPO	Cost: Best -> PPO	Repeated Test: Best -> PPO
0	0.8333	0.8333	1.0000	+0.1667	0.1667 -> 0.0000	8.50 -> 8.50	0.0000 -> 0.0000
1	0.9000	0.9000	1.0000	+0.1000	0.1000 -> 0.0000	8.50 -> 8.50	0.0000 -> 0.0000
2	0.8667	0.8667	1.0000	+0.1333	0.1333 -> 0.0000	8.50 -> 8.50	0.0000 -> 0.0000
Mean	0.8667	0.8667	1.0000	+0.1333	0.1333 -> 0.0000	8.50 -> 8.50	0.0000 -> 0.0000

PAPPO-PPO wins on all three seeds. The mean held-out success improvement over the best non-PPO baseline is 13.33 percentage points, while failed edits fall to zero and average tool cost does not increase.

8.2 Checkpoint Stability

Seed	Update 0	Update 1	Update 2	Update 3	Update 4
0	0.8333	0.8333	0.9000	1.0000	1.0000
1	0.9000	0.9000	0.9000	1.0000	1.0000
2	0.8667	0.9000	0.9000	1.0000	1.0000

The improvement is not a single late checkpoint spike. All three seeds reach 1.0000 held-out success by update 3 and keep it at update 4.

8.3 KL Diagnostics

Seed	Update 0	Update 1	Update 2	Update 3	Update 4
0	0.000000	0.000048	0.000223	0.003136	0.035791
1	0.000000	0.000058	0.000189	0.002219	0.020127
2	0.000000	0.000062	0.000240	0.002476	0.019943

KL remains small through the early improvements and rises modestly by the final checkpoint. This is consistent with a controlled LoRA PPO update rather than an unbounded supervised drift.

9. Interpretation

The earlier weighted-LoRA comparison showed that simply using PAPPO-style scores as supervised weights was too blunt: it often matched or underperformed the base model. The true-PPO loop changes the result. Once the patch-aware turn signal is used as an advantage inside clipped PPO, with old-policy log probabilities, reference control, grouped baselines, cumulative adapters, and fresh rollouts, the method produces a stable held-out

improvement in this setting.

The practical benefit is not only higher success. PAPPO-PPO removes failed edits in the reported held-out split without increasing average tool cost or repeated tests. That matters because a coding-agent training method that improves success by simply spending more tool calls or retrying tests would be less useful.

The current evidence supports the following claim:

> In a controlled RealRepoFix-100 setting with `Qwen/Qwen2.5-Coder-7B-Instruct`, patch-aware turn-local credit assignment becomes useful when embedded in a true PPO loop, improving held-out repair success over non-PPO weighted-LoRA baselines without increasing measured tool cost.

It does not support a broader claim that PAPPO improves every LLM coding agent or every repository-repair benchmark.

10. Reproducibility

The v0.1.0 true-PPO package records the experiment in GitHub and Software Heritage.

- Repository: `https://github.com/TerminusAkivili/PAPPO`
- Release tag: `v0.1.0-trueppo`
- Release URL:

`https://github.com/TerminusAkivili/PAPPO/releases/tag/v0.1.0-trueppo`

- Zenodo DOI:

`https://doi.org/10.5281/zenodo.20971530`

- Zenodo concept DOI:

`https://doi.org/10.5281/zenodo.20971529`

- Core experiment commit:

`5d345d06c735dd263ab0a85f24200c52dd9993e5`

- Software Heritage snapshot:

`swh:1:snp:51a2836ced67734772ba3c5f0bfa69d4b48e1427`

Key files:

- PPO objective and logprob handling: `pappo/ppo_training.py`
- PPO sample extraction: `pappo/ppo_rollout.py`
- Turn/group critic: `pappo/turn_critic.py`
- RealRepoFix task and agent backends:

`pappo/realrepofix.py`, `pappo/llm_agent_pilot.py`

- True-PPO runner: `scripts/run_realrepo_pappo_ppo.py`
- Non-PPO baseline runner: `scripts/run_realrepo_lora_comparison.py`
- Manifest: `data/realrepofix_100_manifest.jsonl`
- Result summary: `docs/pappo_true_ppo_results.md`

The focused verification suite can be run with:

```
python -m pytest -q
```

The final seed-0 experiment can be reproduced with:

```
PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True HF_HUB_DISABLE_XET=1 \
python scripts/run_realrepo_pappo_ppo.py \
  --model Qwen/Qwen2.5-Coder-7B-Instruct \
  --backend hf \
  --manifest data/realrepofix_100_manifest.jsonl \
```

```
  --train-limit 70 \
  --eval-limit 30 \
  --updates 5 \
  --num-rollouts-per-task 2 \
  --max-new-tokens 384 \
  --temperature 0.7 \
  --max-length 768 \
  --learning-rate 1e-4 \
  --ppo-epochs 1 \
  --local-prior 0.25 \
  --no-normalize-advantages \
  --seed 0 \
  --output-dir data/repro_realrepo_pappo_ppo_seed0 \
  --local-files-only
```

Use `--seed 1` and `--seed 2` to reproduce the three-seed result.

11. Limitations

This report intentionally presents PAPPO as a small-scale feasibility result. The current evidence has several limits.

First, the experiment uses one model family and one local benchmark. A stronger claim needs additional coding models and external benchmark families. Second, the held-out set is 30 tasks per seed. The three-seed pattern is encouraging, but it is still a compact evaluation. Third, the current report compares against non-PPO weighted-LoRA baselines; broader RL baselines and independent implementations would make the comparison stronger. Fourth, the method still uses heuristic local credit signals. Those signals are useful here, but future work should test learned critics, richer patch validators, and ablations for the local prior, grouped baseline, KL coefficient, rollout count, and optimized tool set.

Finally, this release tracks code, manifests, reports, and focused tests. Large adapter weights and intermediate rollout dumps are intentionally not committed, so exact byte-for-byte checkpoint reconstruction requires rerunning the experiment with the same model availability and environment.

12. Conclusion

PAPPO addresses a concrete credit-assignment problem in tool-using coding agents: terminal trace rewards are too coarse for repository repair. The released true-PPO implementation turns patch-aware local evidence into turn-level advantages and optimizes generated edit actions with clipped PPO, reference KL control, LoRA adapters, and online rollouts.

The RealRepoFix-100 result is strong enough for a public technical report or arXiv-style preprint if framed carefully. The defensible contribution is a reproducible single-model feasibility finding: PAPPO's turn-local credit signal becomes effective when used inside a true PPO loop, improving held-out repair success without increasing measured tool cost in the released setting.